THE GEORGE WASHINGTON UNIVERSITY

WASHINGTON, DC

Data Science Program

Capstone Report - Spring 2025

An End-to-End Multi Agent AI System for Personal Finance: Synthetic Data Generation, Budget Optimization, and Investment Advisory

Aman Jaglan,
Nemi Makadia,
Smit Pancholi,
Yash Doshi

supervised by
Amir Jafari

Managing personal finances is a complex task, often requiring individuals to budget effectively and make informed investment decisions. Traditional digital finance tools are fragmented and can raise privacy concerns, as they rely on sensitive personal data. This project addresses these challenges by developing an integrated multi-agent financial advisor system powered by large language models (LLMs) and machine learning. The system comprises three key components: (i) a *Synthetic Data Generator* that creates realistic transaction datasets using statistical demographic modeling and generative adversarial networks, enabling model training without exposing real user data; (ii) a *Budget Classification and Optimization module* with an automated transaction classification agent (exploring logistic regression, LSTM, BERT, and few-shot LLM techniques) and a budgeting agent that provides personalized spending recommendations based on the 50/30/20 rule; and (iii) an *Investment Advisory agent* that integrates ensemble sentiment analysis (including a fine-tuned FinBERT model) with an LLM to generate tailored investment advice. We detail the design and implementation of these components and evaluate their performance. Our results show that the fine-tuned BERT model achieves around 71% classification accuracy (macro-F1 0.62) in categorizing expenses, the budgeting agent produces sensible suggestions to align spending with goals, and the LLM-based investment advisor can synthesize market data and sentiment into coherent investment recommendations. The integrated advisor operates in near real-time and preserves privacy via synthetic data. These findings demonstrate the feasibility of a comprehensive AI-driven financial advisor that combines data generation, predictive analytics, and natural language generation to assist users in personal finance management.

Contents

List of Figures

List of Tables

1 Introduction

Personal finance management involves budgeting expenditures and making investment decisions, tasks that can be daunting for many individuals. In recent years, fintech applications and robo-advisors have emerged to help people manage money, but these typically address isolated aspects (either budgeting or investments) and often require users to share sensitive financial information. Ensuring data privacy while providing intelligent, personalized financial guidance remains an important challenge. Advances in artificial intelligence, particularly large language models (LLMs) and machine learning in finance, create opportunities for more integrated solutions. For example, modern AI can analyze spending patterns, predict market trends, and even generate human-like advice tailored to a user's situation.

In this project, we develop an end-to-end **LLM-based Financial Advisor Agent** that combines budgeting assistance with investment advice in one system. The system is designed with a modular multi-agent architecture comprising distinct but interconnected components: a synthetic data generation module, a budget classification and optimization agent, and an investment advisory agent. By generating realistic synthetic financial data, we avoid using actual personal data and thus address privacy concerns while still training and testing our models on lifelike scenarios. The budgeting module automatically categorizes transactions and suggests adjustments to meet financial goals (like maintaining a balanced budget), and the investment module leverages sentiment analysis and LLMs to provide stock portfolio recommendations and market insights.

The main objectives of our work are to: (1) create a realistic synthetic dataset to fuel data-driven financial analysis without compromising privacy; (2) achieve accurate classification of expenses into meaningful categories and provide optimal budget allocation suggestions; (3) integrate financial sentiment analysis and predictive modeling to inform an LLM-driven investment advisor capable of generating comprehensive advice. We report on the development of each component and evaluate their performance. Our integrated agent demonstrates that AI techniques can be successfully applied to personal finance, yielding a tool that helps users understand and improve their financial health. In summary, this report details our approach and findings in building a novel AI-based personal financial advisor that merges budgeting and investment guidance into a single coherent system.

To provide an overview of our end-to-end platform, we present the complete system architecture in Figure 1. The diagram illustrates how the three major components—synthetic data generation, budget classification and optimization, and investment advisory—are integrated into a seamless pipeline. It details the flow of inputs, including user transactions, demographic information, and market data, as well as the interactions between modules and the generation of outputs such as budgetary recommendations and personalized investment advice.

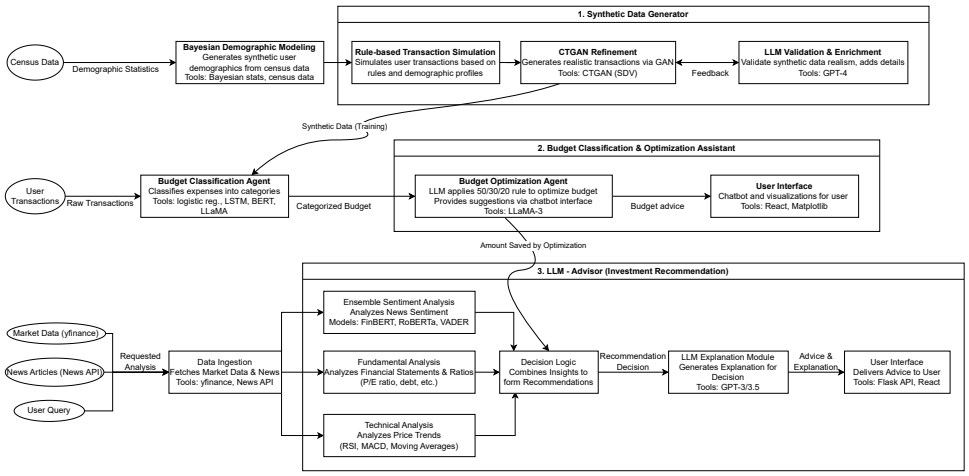

Figure 1: Overall System Architecture: Integration of Synthetic Data Generation, Budget Classification and Optimization, and Investment Advisory Agents.

2 Problem Statement

Managing personal finances is a critical yet challenging task for individuals, often hindered by limited access to high-quality transactional data, a lack of sophisticated budgeting tools, and an absence of personalized investment advisory services. Traditional financial management platforms either rely heavily on proprietary user data—raising privacy concerns—or provide generic advice without incorporating advanced AI capabilities. Furthermore, budget optimization solutions frequently lack personalization, failing to adapt to diverse income levels and spending behaviors, while existing robo-advisory systems often exclude unstructured data analysis such as real-time sentiment from news or social media.

There is an evident gap for an end-to-end AI-driven system that can: (1) synthesize realistic, privacy-preserving financial transaction datasets to facilitate model development; (2) automate accurate transaction classification and budgeting advice tailored to individual users; and (3) integrate real-time financial sentiment analysis, fundamental evaluation, and technical indicators to deliver explainable and personalized investment recommendations.

This project aims to address these gaps by developing a comprehensive, multi-agent AI system for personal finance management. The system encompasses synthetic data generation using Bayesian models and GANs, intelligent budget classification and optimization via transformer-based models and large language models (LLMs), and an LLM-empowered investment advisor capable of transparent, real-time decision-making. The overarching objective is to enhance financial literacy, promote better budgeting, and empower users to make informed investment choices—all while safeguarding data privacy.

3 Related Work

There is a rich body of literature on applying AI to financial data, which informs our work. In the area of *synthetic financial data generation*, researchers have developed methods to simulate realistic transaction data for model training and testing. Generative models like the Conditional Tabular GAN (CTGAN) have been used to model tabular financial records with high fidelity. Xu *et al.* introduced CTGAN, which can capture complex relationships in tabular data and generate synthetic samples that mimic real datasets. In the financial domain, Lopez-Rojas *et al.* developed PaySim, a simulator for mobile money transactions to facilitate fraud detection research. More recently, Mehri *et al.* proposed BankGAN, a GAN-based approach specifically for banking transaction data, demonstrating improved realism in synthetic financial transactions. These works show that GANs and other generative models can produce convincing financial data, although ensuring that synthetic data reflects real-world variety and edge cases remains challenging. Our approach builds on these ideas by using a multi-stage generation process (combining statistical modeling and GANs) to create both user demographic profiles and detailed transaction records.

For the task of *transaction categorization*, earlier studies applied classical machine learning and deep learning to classify expenses. Hossain and Dustdar (2019) employed supervised learning algorithms to categorize transactions and reported that feature engineering and data quality significantly impact accuracy. Recent advances leverage natural language processing for transaction descriptions. Pre-trained language models like BERT have achieved success in text classification tasks across domains, including finance, due to their contextual understanding of language. FinBERT, a BERT variant pre-trained on financial text, has demonstrated improved sentiment analysis performance on financial news, indicating that domain-specific language models can be advantageous for financial NLP tasks. Moreover, large language models such as GPT-3 have shown the ability to perform tasks with minimal examples (few-shot learning) by leveraging vast pre-trained knowledge. Brown *et al.* famously showed that GPT-3 can learn new tasks from a handful of examples provided in its prompt, without additional fine-tuning. This few-shot capability suggests that an LLM might classify transactions or provide financial advice by understanding contextual cues, even with limited direct training on the task. Indeed, emerging LLMs in the finance domain like BloombergGPT (a 50-billion parameter model trained on financial data) have outperformed general models in financial NLP benchmarks, underscoring the value of specialized large models for tasks like sentiment analysis and report generation.

In the realm of personal budgeting and financial planning, a well-known guideline is the *50/30/20 rule*, popularized by Warren and Tyagi. This rule-of-thumb suggests allocating 50% of income to needs (essentials), 30% to wants (discretionary spending), and 20% to savings or debt repayment. While simplistic, this framework is widely cited for basic budgeting advice and provides a starting point for automated budgeting agents. Modern budgeting tools and research often build on this concept to tailor budget allocations to individual circumstances. However, a challenge is that strict rules may not fit everyone; thus personalized recommendations (for example, suggesting where specifically to cut

expenses) can make such guidelines more actionable. Large language models could be leveraged here to translate numeric budget analyses into user-friendly advice, bridging the gap between raw financial metrics and practical suggestions.

For *investment advising*, our work connects to literature on sentiment-informed trading strategies and LLM-based decision support. Sentiment analysis of news and social media has been studied as a predictor of stock market movements. Tools like VADER provide rule-based sentiment scoring suitable for social media text, while FinBERT and other transformer-based classifiers can capture nuances in financial news sentiment. Prior studies found that incorporating news sentiment can improve the performance of predictive models for stock price movements. Furthermore, large-scale language models have started to be used for generating financial insights. For instance, researchers have explored prompting GPT-3 to act as a financial advisor or to interpret financial reports, taking advantage of its general world knowledge. The introduction of finance-specific LLMs such as BloombergGPT illustrates a trend towards LLMs that can deeply understand financial context and jargon. Our investment advisor agent is inspired by these developments: it uses specialized NLP (financial sentiment analysis and stock movement classification) in tandem with an LLM to produce narrative advice. To our knowledge, integrating a fine-tuned financial sentiment model with a generative LLM for real-time personalized investment recommendations is a novel approach, as most existing robo-advisors rely on either purely quantitative models or static questionnaires. By combining multiple AI techniques, our system aims to emulate a human financial advisor who analyzes data and communicates advice in natural language.

4 Solution and Methodology

Our integrated financial advisor consists of multiple components working together. Figure **??** (not shown here) provides an overview of the system architecture, which includes: (1) a Synthetic Data Generation module, (2) Budget Classification and Optimization agents, and (3) an LLM-based Investment Advisor agent. In this section, we describe the design and techniques used for each component in detail.

4.1 Synthetic Data Generation

Real transaction data is often private and difficult to obtain for research. To overcome this, we developed a multi-stage synthetic data generation system to create a realistic dataset of personal financial transactions. The goal was to produce data that capture the distribution of incomes, spending habits, and merchant patterns one would expect in a real population, thus enabling training of our models in a privacy-preserving manner.

Demographic Modeling: In the first stage, we generated synthetic customer profiles by leveraging real demographic statistics as priors. We used public data from the U.S. Census and local government sources for Washington, D.C. to inform this process. Specifically, we obtained population demographics by ZIP code (including distributions of age, household income, household size, etc.) from the American Community Survey. Using these statistics, we sampled attributes for synthetic individuals. For example,

for each synthetic person we randomly assign an age, income, and household size that reflect the probability distributions of a given ZIP code. We also assign attributes like marital status and gender. A simple Bayesian approach was used to preserve correlations: for instance, income level is sampled conditional on age (older individuals tend to have higher income) and household size is correlated with marital status. By using conditional probabilities derived from real data, the synthetic population maintains plausible relationships between attributes (e.g., a 22-year-old single student is likely to have a lower income and smaller household size than a 45-year-old married person). We generated a population of several thousand synthetic customers across various DC ZIP codes, providing a diverse foundation for financial behaviors.

We began with a detailed demographic study of the target region. Specifically, Zipcode 20001 in Washington, DC was chosen due to its diversity and data availability. The data was sourced from the U.S. Census Bureau and DC Open Data Portal.

Table 1: Demographic Snapshot for Zipcode 20001

Feature	**Value**
Total Households	23,831
Households Age 25–44	14,460
Married-couple Families	4,860
Female Householders (no spouse)	1,562
2-person Families	4,320
1 Earner Households	1,263
White Householders (%)	54.5
Black Householders (%)	26.8

Merchant and Category Mapping: To simulate realistic transactions, we needed an inventory of merchants and categories. We utilized the District of Columbia Open Data portal, which includes an "Active Business Licenses" dataset listing businesses operating in DC along with their industry classifications. From this, we extracted a list of common merchant names and associated categories (e.g., restaurants, grocery stores, retail shops, travel services). We curated a set of spending categories relevant to personal finance, such as *Groceries*, *Dining*, *Utilities*, *Transportation*, *Entertainment*, *Travel*, *Health*, *Rent*, etc., and mapped each business to one of these categories. This mapping allowed us to randomly pick realistic merchant names and categories for synthetic purchases.

Transaction Generation via GAN: In the second stage, we generated financial transactions for each synthetic customer. We employed a generative adversarial network approach, specifically the Conditional Tabular GAN (CTGAN) by Xu *et al.*, to model the distribution of transaction data. We assembled a seed dataset to train the GAN by combining patterns from public financial datasets and domain knowledge. This seed data included records with attributes: *UserID*, *Date*, *Merchant*, *Category*, *Amount*, and *Payment Type*. For example, we incorporated example spending patterns (from personal finance examples and open-source financial transaction data, when available) ensuring

that the GAN had some grounding in reality. CTGAN was trained with *Category* as a conditional input, so that it could generate transaction amounts conditioned on the type of expense. This is important because different categories have different price distributions (e.g., grocery transactions might typically be tens of dollars, whereas rent payments are thousands). The GAN learned the joint distribution of transaction features. After training, we used the generator to produce a large number of synthetic transactions for each category.

Using the demographic distributions, a GAN model was trained to generate sample customer profiles. These profiles represent realistic individuals based on zip codes, age, income, household type, and gender.

Table 2: Sample Customers Generated by GAN

ID	Zip	Age	Status	Size	Income	Gender
0bed...376a	20019	62	Single	2	$56,221	Female
d0a1...87ce	20010	24	Single	3	$76,289	Female
4076...7e12	20009	61	Married	3	$144,933	Male

CTGAN enabled us to model the joint distribution of transaction features, such as merchant name, transaction category, and amount. Example outputs are shown in Table 3.

Table 3: Sample Transactions Generated by CTGAN

User ID	Zip	Category	Merchant	Amount ($)
2478...4294c	20006	Groceries	Vidol Luffon LLC	360.62
dbc1...fd99	20004	Dining	Jasmines Hair Gallery	148.09
3f83...e03e04	20001	Personal Care	CHURCH DC LLC.	167.71

Each synthetic customer was then assigned a set of transactions. We did this by first sampling how many transactions the customer makes in a given period (we drew monthly transaction counts from a distribution based on their income and spending propensity). Then for each transaction needed, we sampled a category (weighted by typical spending breakdowns for that income level), and used the GAN to generate a transaction (amount and other details) of that category. The merchant name was randomly picked from the list of merchants corresponding to that category (ensuring geographical plausibility by biasing towards merchants in the customer's city). Dates for transactions were spread across the months of a year. Payment type (credit, debit, cash) was randomly assigned with a bias towards credit/debit for most purchases and perhaps cash for small amounts.

By this two-step process, we created a comprehensive *synthetic transaction dataset* with thousands of customers and their detailed transaction histories. Our synthetic data generation is *multi-stage* in that it combines a statistical model for user demographics with a GAN for transaction details. We also experimented with alternative generation methods for comparison. For instance, we tried using a large language model (GPT-4 via API) to generate transaction descriptions and amounts given a prompt describing a

person's profile (e.g., "Generate 10 transactions for a 30-year-old in DC earning $70k: include dates, merchants, categories, and amounts"). The LLM-based approach produced reasonably realistic transactions with rich descriptions, but it lacked consistency in numeric distributions (e.g., sometimes generating implausibly large or small expenses). Ultimately, our hybrid approach allowed precise control over numeric realism (through the GAN) while still reflecting real-world entities (via the merchant catalog).

Our hybrid model combined the tabular strengths of CTGAN and the narrative versatility of LLMs. The result was semantically rich transactions rooted in statistical realism.

Table 4: Sample Transactions Generated by Hybrid Model

Amount ($)	**Date**	**Merchant Name**	**Category**	**Payment Type**
43.92	2024-03-16	SIAM HOUSE DC INC.	Restaurant Dining	Credit Card
23.50	2024-03-16	APRA ONE CORP	Restaurant Dining	Credit Card
85.99	2024-11-20	UNIVERSITY OF THE DISTRICT OF COLUMBIA (UDC)	Catering Services	Credit
145.67	2024-11-20	Target Corporation	Grocery Store	Credit Card

The final synthetic dataset was evaluated qualitatively and statistically. Key summary statistics (average monthly spend per category, distribution of transaction amounts, etc.) were examined to ensure they fell within realistic ranges. For example, in our generated data the average monthly essential spending (needs) was about 52% of income, close to the 50% target, and the distribution of grocery bills or restaurant charges aligned with known averages. We also verified that the synthetic dataset contained edge cases like occasional very large purchases or sparse spending individuals, albeit these were generated with lower probability. Overall, the synthetic data served as a viable stand-in for real personal finance data, supporting downstream model training while upholding privacy.

4.2 Budget Classification and Optimization Agents

This module of our system takes as input a set of transactions (whether from a real user or from our synthetic dataset) and the user's income, then outputs two things: a categorized list of transactions (each labeled with a spending category) and personalized budget advice for the user.

4.2.1 Budget Classification Agent

The budget classification agent is responsible for automatically assigning a spending category label to each transaction record. This is a multi-class text classification problem, where the primary feature available is the transaction description (often containing the merchant name) and potentially the transaction amount. For example, a transaction

description "Starbucks Cafe" should be classified as *Dining* or *Coffee Shop*, whereas "United Airlines FLT 802" should be *Travel*. Accurate classification is essential for building a clear picture of where the user's money is going.

We approached this task by experimenting with four different modeling techniques, reflecting an evolution from traditional methods to state-of-the-art language models:

- **Logistic Regression (Baseline):** As a baseline, we implemented a logistic regression classifier using TF-IDF features extracted from transaction descriptions. Each description was transformed into a vector of word feature weights, and a one-vs-rest logistic regression was trained to predict the category. This model is fast and interpretable; certain keywords strongly weight towards particular categories (e.g., "Airlines" for Travel, "Hospital" for Health). However, being a linear model, it cannot capture context beyond individual words or slight phrasing variations.

- **LSTM Neural Network:** Next, we built a recurrent neural network model using an embedding layer and a Long Short-Term Memory (LSTM) layer to process the sequence of words in each transaction description. The LSTM can capture word order and context (for instance, distinguishing "Bank of America Credit Payment" as a financial transfer vs. just "America" which has no meaning alone for category). Our LSTM model was implemented in PyTorch. We used pre-trained GloVe embeddings to initialize the embedding layer so that financial terms would have a reasonable vector representation. A dropout layer and a dense output layer with softmax provided category probabilities. We trained the LSTM on the synthetic labeled transactions, using a weighted cross-entropy loss to address class imbalance (some categories like *Groceries* were far more frequent than others like *Travel*). Despite tuning hyperparameters, this model proved tricky to optimize, possibly due to limited training data for the rarer classes.

- **Fine-tuned BERT:** We leveraged the transformer-based model BERT for this classification task, hypothesizing that its deep bidirectional language understanding would yield higher accuracy. Specifically, we used the *bert-base-uncased* model and fine-tuned it on our transaction data. Each transaction description was fed to BERT, and we added a classification layer on top of the `[CLS]` token representation. We took advantage of the HuggingFace Transformers library for training, using their Trainer API with a custom compute metrics function to calculate accuracy and macro F1 score per epoch. To further mitigate class imbalance, we extended the model to *WeightedBertForSequenceClassification*, applying higher loss weights to underrepresented classes. Fine-tuning was done for a few epochs (we found 3 epochs at a 2e-5 learning rate sufficient) given the relatively small input text length and dataset size. We also experimented with a variant using FinBERT (a BERT model pre-trained on financial texts) expecting it might better handle merchant names and financial terms, but initial results were similar to the generic BERT, likely because merchant names are proper nouns not present in FinBERT's training corpus either.

- **Few-shot LLM (LLaMA-3):** Lastly, we explored an approach using a large language model in a few-shot setting, thus avoiding explicit model training. We accessed a hosted API for a LLaMA-3 model with 8 billion parameters (offered by Groq Inc.). This model has a context length of 8192 tokens, enabling us to provide several examples of categorized transactions in a prompt, and then ask it to categorize new transactions. We constructed prompts containing 5-6 examples like "`[Description] -> [Category]`" and then a test description, instructing the model to continue the pattern. This approach leverages the LLM's powerful comprehension and was very convenient as it required no training time. We expected the LLM's extensive pretraining (which likely included some knowledge of common merchants and categories) to generalize well.

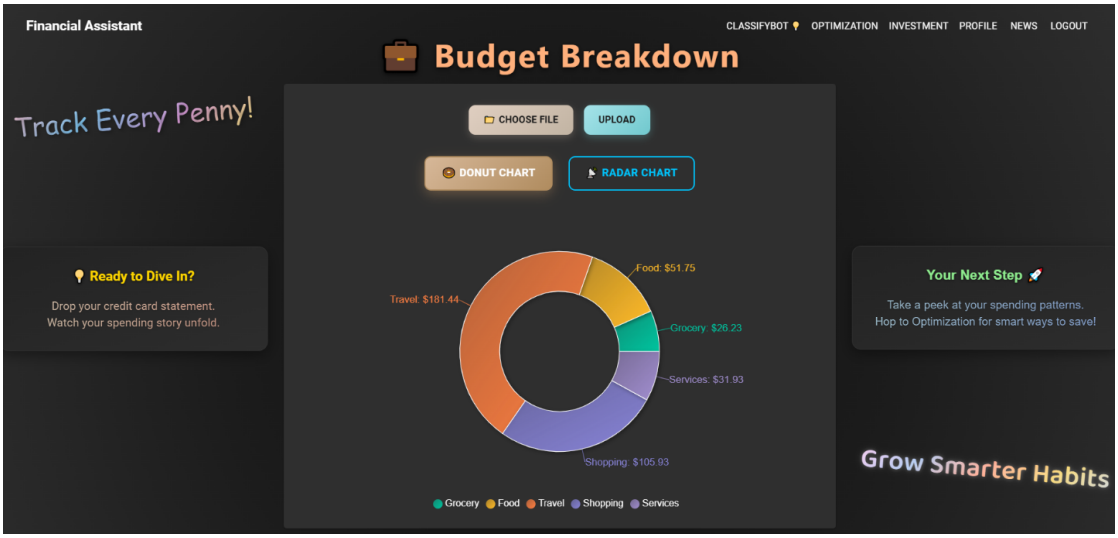

Figure 2: Classified Spending Breakdown (Bank of America)

The LLaMA3 model was evaluated using a human-in-the-loop framework due to the lack of ground-truth labels in the API-based few-shot classification setup. Model responses were assessed qualitatively across various real and synthetic transaction samples. Human reviewers manually verified the correctness of predictions, and the model consistently demonstrated high accuracy and adaptability to ambiguous or rare merchant names (e.g., peer-to-peer apps, airlines).

While traditional metrics such as accuracy or macro-F1 were not computable in this API-only integration, we observed consistent correctness across diverse test cases, making it the most reliable and generalizable model in our pipeline. Given its strong zero-shot performance, minimal setup, and ease of use, the LLaMA3-based approach was selected for production deployment.

However, one limitation observed was occasional inconsistency in classification for the same merchant across different runs. When the same bank statement was uploaded twice, the model sometimes assigned different categories to the same merchant. Although

predictions remained semantically reasonable, this highlights the stochastic nature of prompt-based large language models. Despite this, overall performance remained highly accurate and aligned with human expectations in most cases.

After training or implementing each model, we evaluated their performance on a reserved test set of synthetic transactions with known true categories. We used Accuracy and Macro F1-score as the key metrics. Accuracy simply measures overall percentage of correct labels, while Macro F1 provides an aggregate of per-class precision and recall, treating all categories equally regardless of frequency. Macro F1 is important here because our dataset (like real spending) is imbalanced; for example, *Food* and *Housing* might comprise a large fraction of transactions, whereas *Travel* or *Education* purchases are less frequent. A model could achieve high accuracy by doing well on common classes only, but a good macro F1 indicates it handles even the rare classes reasonably.

Model	**Accuracy**	**Macro F1**
Logistic Regression	∼71%	0.57
LSTM (RNN)	∼39%	0.40
BERT (fine-tuned)	∼72%	0.62
LLaMA-3 (few-shot)	N/A	N/A

Table 5: Comparison of models for transaction category classification. The LLaMA-3 few-shot model was evaluated qualitatively (no single accuracy value).

Table 5 summarizes the performance of the different classification approaches. The logistic regression baseline, despite its simplicity, achieved an overall accuracy of about 71%. Many transactions have distinctive keywords (e.g., "Airlines", "Gas", "Market") that the model can latch onto, which explains the reasonably high baseline. However, its macro F1 of 0.57 indicates it struggled on minority categories (often predicting the majority class for those cases). The LSTM model underperformed significantly, with under 40% accuracy. We suspect that the LSTM might have overfit the training data or not been sufficiently complex; the limited size of our training set and high dimensionality could have made training difficult. The fine-tuned BERT model also achieved about 71% accuracy, but importantly, it improved macro F1 to 0.62. This implies BERT was better at pulling out context and correctly identifying less frequent categories compared to logistic regression. For example, BERT could differentiate "Hilton Garden Inn" (a hotel, hence Travel) from "Hilton Grocery" (if that existed, as a hypothetical store name) by subtle context, whereas logistic regression might misclassify purely based on seeing "Hilton". The BERT classifier provided a good balance of precision and recall across classes, making it the best performing trained model in our comparison.

The few-shot LLM (LLaMA-3) did not produce a single numeric accuracy since we did not label a large set of its outputs due to API usage costs. Instead, we performed a qualitative assessment with a sample of transactions. Impressively, the LLM classified most examples correctly, including some tricky ones involving less obvious merchant names. For instance, given "BLK*Spotify" (a cryptic description for a music streaming

subscription), the LLM correctly inferred it as an Entertainment expense. In another example, for "Payment to AMEX", it deduced it's a credit card bill payment (which we categorized under *Financial*). These qualitative results suggest the LLM's accuracy is quite high, likely on par with BERT or better, but we also observed a few mistakes (e.g., it misclassified a local city parking authority fee as *Shopping* instead of *Transportation*). The LLM's advantage is its ability to interpret unseen or rare strings by drawing on broad knowledge (perhaps recognizing "AMEX" as American Express). However, reliance on an external API-based model raises issues of cost and latency for large-scale use. In our case, the few-shot model served as a proof of concept that large pretrained models can generalize the categorization task with minimal examples.

Based on these findings, we chose the fine-tuned BERT model as the primary classification engine in our system due to its strong and balanced performance and the practicality of running it locally. The classification agent thus takes each new transaction description as input and outputs a category label. These labeled transactions are then fed into the budgeting algorithm.

4.2.2 Budget Optimization Agent

Once transactions are categorized, the next step is to help the user understand their budget and find ways to optimize it. Our budget optimization agent aggregates the classified transactions, compares the user's spending against income and recommended targets, and generates personalized suggestions for improvement.

Firstly, we group the classified transactions into high-level budget buckets. We broadly follow the 50/30/20 rule, dividing expenses into Needs, Wants, and Savings/Debt. To do this, we maintain a mapping of detailed categories to these buckets. For example, categories like *Rent*, *Utilities*, *Groceries*, *Insurance*, and *Healthcare* are mapped to **Needs** (essential spending). Categories such as *Dining Out*, *Entertainment*, *Shopping*, *Travel* are mapped to **Wants** (discretionary spending). Any explicit *Savings*, *Investments*, or *Debt payments* are mapped to the **Savings/Debt** bucket. If the user does not explicitly list savings (which is common, as many simply have leftover income as implicit savings), we calculate savings as: *Savings = Income - (Needs + Wants)* for that period. This way, every dollar of net income is allocated to one of the three buckets.

After aggregation, we compute the percentage of income going to each bucket. Suppose a user's monthly net income is \$5000. If \$3000 went to Needs, \$1500 to Wants, and \$500 remained (Savings), then the splits are 60%, 30%, 10%. We then compare these to the ideal 50%, 30%, 20% benchmark. In this example, Needs are overshooting by 10 percentage points, and Savings are only half the recommended level. These deviations form the basis of our optimization suggestions.

The agent uses straightforward arithmetic to recommend adjustments: reduce the overspent areas and increase the underspent (particularly savings). In the above scenario, it might recommend cutting Needs by 10% of income (which is \$500) and reallocating that to savings. However, not all Needs can be easily cut (rent is fixed, for instance). So our agent drills down into subcategories to identify where there is flexibility. If the user's Needs are high, we check if any subcategory is above typical benchmarks — e.g.,

maybe Groceries are $800 (could potentially be trimmed) or Utilities are exorbitant due to an inefficient plan. If Wants are high, we see which discretionary category dominated (maybe Dining Out is $1000, far above average).

To make concrete suggestions, we leverage an LLM to generate the advice narrative. We pass to the LLM a structured summary of the user's budget: for example, "This user spends 60% of income on needs (recommended 50%), 30% on wants (recommended 30%), and saves 10% (recommended 20%). They spend $1000 on dining out and $500 on entertainment per month." Along with this, we include guidance text like "Suggest how they can reduce spending on wants or certain needs, and increase savings. Be specific and encouraging." Using the Groq LLaMA-3 model via API (the same model we tried for classification, but here used for generation), the agent produces a set of suggestions. Because the LLM has understanding of everyday life, it can create meaningful and tailored advice. For instance, it might output: *"Consider reducing dining out to once a week instead of three times, which could save around $400 per month. Try cooking at home more often and setting a firm grocery budget. Additionally, review your subscription services under entertainment – you might cancel ones you use infrequently, saving perhaps $50 a month. Redirect these savings into an automatic transfer to a savings account at the start of each month to help reach the 20% goal."*

The use of the LLM here brings in an element of empathy and nuance that a rule-based system would lack. Rather than just stating "cut $500 from wants," it provides context and actionable ideas (like "once a week dining out" or "cancel unused subscriptions"). We found that this approach yields advice that sounds much like what a human financial advisor might say, making it more digestible for users.

The Budget Optimization Agent was evaluated using categorized spending data and an income of $24,500. The agent analyzed spending patterns following the 50/30/20 financial rule, categorizing expenses into "Needs", "Wants", and "Savings" groups. Based on this analysis, it identified the Grocery and Services categories as areas with the greatest potential for cost reduction.

Figure 3 displays the natural language suggestions generated by the agent for overspending categories, while Figure 4 shows the categorized spending cards and estimated savings summary.

Key findings include:

- **Grocery Suggestions:** Recommended shopping at more affordable stores (e.g., Aldi, Lidl), planning meals in advance, and buying in bulk. Projected savings: $50–$75 per month.
- **Services Suggestions:** Recommended canceling non-essential subscriptions, negotiating with providers, and using DIY alternatives. Estimated savings: $20–$30 per month.
- **Combined Savings:** Overall projected monthly savings ranged between $70–$105, with an average estimated savings of $87.50 highlighted in the user interface.
- **Budget Health Overview:** Each spending category (e.g., Grocery, Food, Travel,

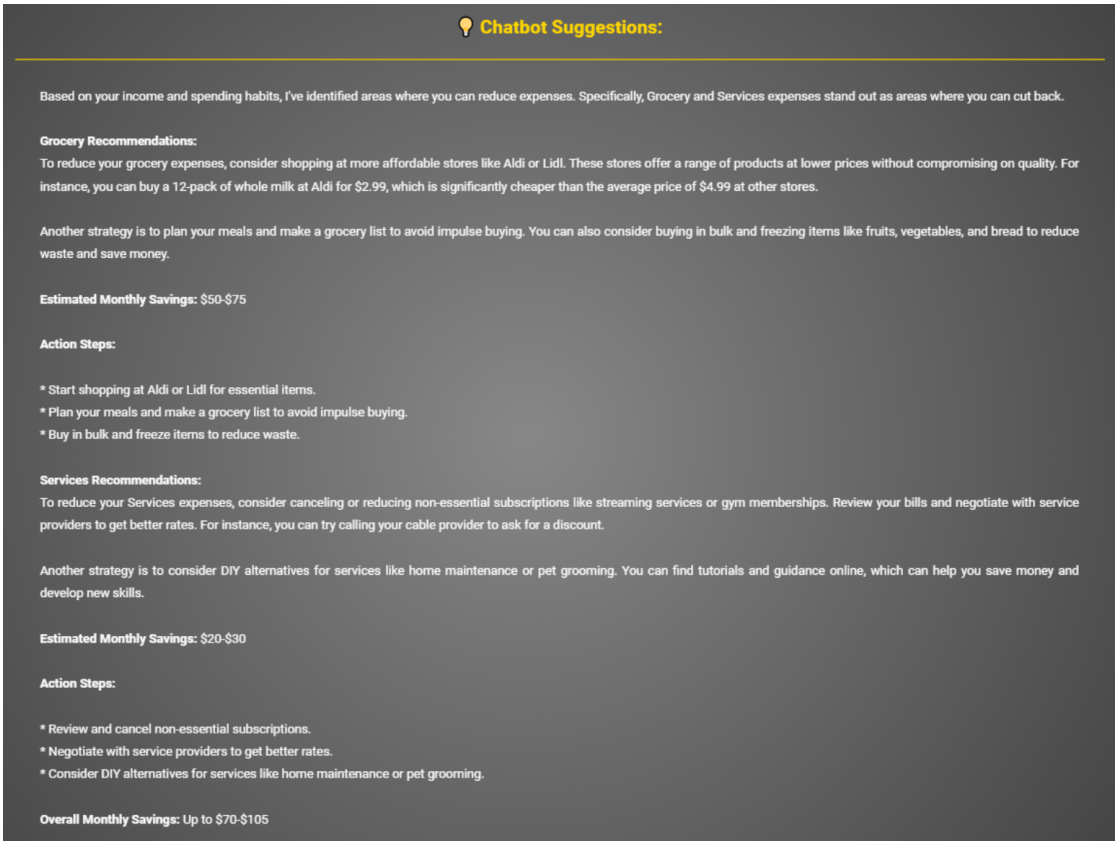

Figure 3: Optimization Output with Suggestions and Savings Summary.

Shopping, Services) was labeled as "Under Control" relative to income thresholds, indicating successful budget optimization.

These evaluations confirmed that the optimization agent effectively:

- Accurately identified target areas for financial improvement.
- Generated actionable and personalized savings recommendations.
- Delivered dynamic, natural language feedback with quantifiable results.

The suggestions from the optimization agent are finally compiled into a brief report for the user. It typically includes a recap of their current budget allocation vs recommended, followed by the suggestions list. In terms of implementation, this agent runs on-demand (for example, when the user finishes a month or requests a budget review). It does not involve model training; instead, it applies a formula and prompt-based generation. We did test a purely programmatic approach (pre-defining suggestion templates for certain conditions), but the LLM-generated suggestions were noticeably richer and more personalized, so we adopted the latter as our default.

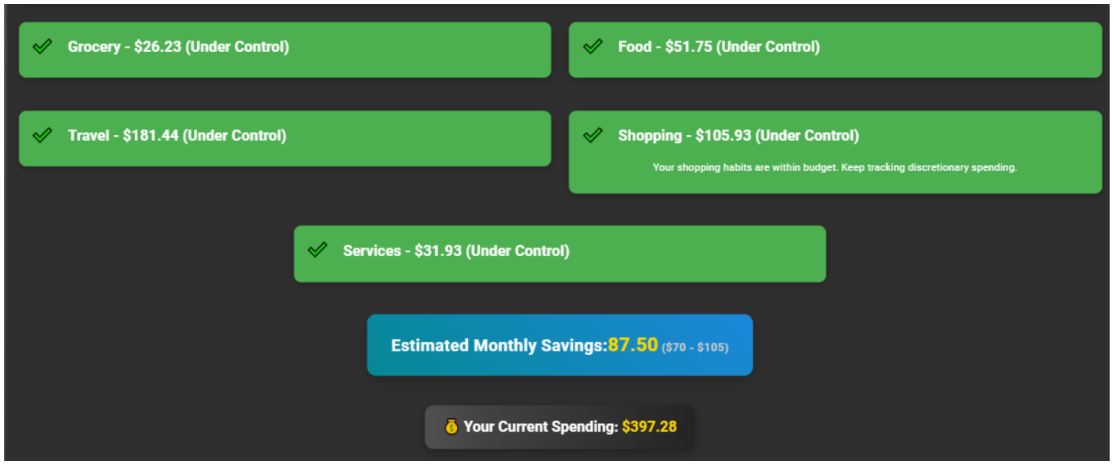

Figure 4: Category-Wise Spending Cards and Estimated Monthly Savings.

4.3 Investment Agent Development

The investment advisor agent (which we call the *LLM-Advisor*) is responsible for providing personalized investment insights and recommendations. It combines analytical components (data fetching, sentiment and predictive analytics) with generative components (explanation and advice via an LLM).

The workflow of this agent is as follows:

1. **Data Retrieval:** When a user is interested in a particular stock or portfolio, the agent first gathers up-to-date information. We integrated the Alpha Vantage API to fetch real-time stock quotes and historical market data for specific symbols (e.g., latest price, recent trends). In addition, to gauge market sentiment, we fetch recent news headlines related to the target company or the market using the NewsAPI service. For example, if the user asks about investing in Tesla, the agent will retrieve the latest news articles about Tesla or the EV industry.

2. **Sentiment Analysis:** The agent processes the fetched news to determine the sentiment or tone of the recent coverage. We utilize FinBERT, a financial sentiment model, to classify news headlines or summaries as positive, negative, or neutral with respect to the stock. FinBERT, introduced by Araci (2019), is pre-trained on financial text and has been shown to outperform general sentiment models for financial news. In our implementation, we fine-tuned FinBERT on a small set of labeled news headlines (annotated whether the news was likely to make the stock go up or down) to tailor it to stock movement sentiment specifically. Additionally, for robustness, we compare its output with a simpler baseline sentiment analyzer (VADER). This ensemble approach helps ensure that no extreme sentiment is missed—if both FinBERT and VADER flag news as very negative, we take that as strong evidence of negative sentiment. In practice, FinBERT provides nuanced results (e.g., identifying a headline like "Company X beats earnings expectations but

lowers future guidance" as neutral or mixed, whereas VADER might get swayed by words like "lowers" and rate it negative). Overall, the sentiment analysis component produces a summary such as "Recent news sentiment for Tesla is mixed to positive."

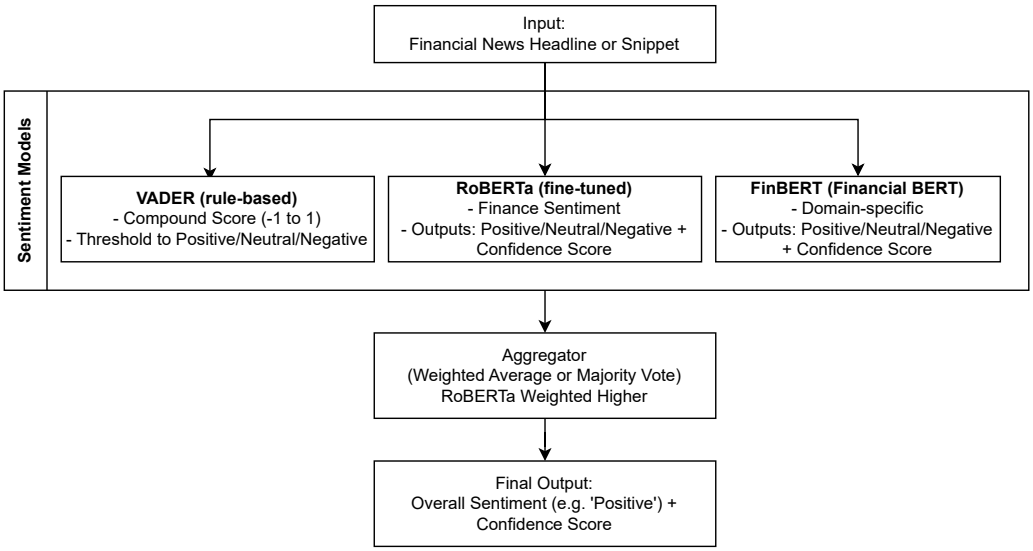

Figure 5: Architecture of the Ensemble Sentiment Analysis Model combining FinBERT, RoBERTa, and VADER outputs using a weighted aggregation strategy.

Table 6: Example Sentiment Analysis Output: Tesla News Headlines Evaluated by the Ensemble Model.

Headline	**Score**	**Sentiment**
Tesla's fight for survival: Why Americans are abandoning the brand and Elon Musk	-0.63	Negative
Donald Trump shares thoughts on Elon Musk's DOGE step back	0.06	Neutral
JPMorgan Chase & Co. has lowered expectations for Tesla stock price	-0.53	Negative
Robert W. Baird cuts Tesla stock price target to $395	-0.06	Neutral

As an illustrative output of this module, Table 6 presents a sample sentiment analysis breakdown for Tesla-related headlines processed by our ensemble of FinBERT, RoBERTa, and VADER. This output highlights the advisor's ability to synthesize raw financial news into consistent and explainable sentiment signals.

Table 7 shows a side-by-side comparison of sentiment classifications across individual models and the final ensemble result. This comparison emphasizes how the

Table 7: Comparison of Sentiment Labels Across Models for Sample Financial News Headlines.

News Headline	**FinBERT**	**RoBERTa**	**Ensemble**
Apple beats earnings expectations, iPhone sales rebound	Positive	Positive	Positive
Ford warns of EV margin pressures in upcoming quarter	Negative	Neutral	Negative
Amazon expands grocery delivery across U.S. cities	Neutral	Positive	Positive
Netflix faces subscriber loss after price hikes	Negative	Negative	Negative
Google's AI division sees modest growth amid tough competition	Neutral	Neutral	Neutral

ensemble strategy corrects edge cases where single models may fail — for example, correctly marking the Amazon expansion as Positive despite FinBERT's Neutral prediction.

This example illustrates several key takeaways:

- FinBERT typically excels in domain-specific sentiment.
- RoBERTa offers general sentiment understanding but may under- or overestimate tone.
- The ensemble aggregates model outputs to yield a balanced and more reliable interpretation.

3. **Stock Trend Prediction:** Apart from qualitative sentiment, our agent attempts to quantitatively predict near-term stock movement. We developed a stock movement prediction model that uses historical data and technical indicators to forecast whether a stock's price will go up or down in the next trading day. Rather than designing a model from scratch, we repurposed NLP models (FinBERT and DeBERTa) by formatting technical data as a textual input. We created a dataset of daily stock information over the past 10 years for a selection of S&P 500 companies (covering various sectors). Each data sample included technical indicators such as the previous day's closing price, volume, moving averages, volatility index, etc., along with the actual outcome (next day up or down). We then represented each sample as a text string (for example: "Yesterday: price $150, volume 10M, 5-day moving average $148, RSI 55. Next day: [Up/Down]"). FinBERT and DeBERTa (a state-of-the-art transformer from Microsoft with improved encoding of text) were fine-tuned on this dataset as sequence classification tasks. Essentially, the models learned to read these "technical reports" and predict up vs down. We found that DeBERTa (large model, 1.5B parameters) showed slightly better performance than FinBERT on this task (likely due to its greater capacity and the fact that the input is a structured pseudo-language rather than real natural text) – detailed results are reported in Section 5. Nevertheless, FinBERT was still useful due to its integration with sentiment. We ultimately included both: the trend predictor component uses the fine-tuned DeBERTa model to get a prediction (e.g.,

"Prediction: Up with 57% confidence") and also considers the FinBERT model's prediction if available (as a second opinion).

4. **LLM-driven Advice Synthesis:** The final and key step is synthesizing all this information into advice for the user. We employed OpenAI's GPT-3.5 (via API) as the generative brain of the investment advisor. We construct a prompt that feeds in the relevant data and asks for a comprehensive recommendation. An example prompt structure is:

```
Stock: Tesla (TSLA)
FinBERT Prediction: Up (confidence 0.85)
DeBERTa Prediction: Up (confidence 0.60)
Latest Price: $750
Recent News Summary: Tesla reported record sales this quarter, though
supply chain issues continue. Analysts remain cautiously optimistic.
User Profile: moderate risk tolerance; $5000 available to invest.
###
As an AI financial advisor, provide a detailed investment thesis for the
above stock. Include analysis of financial performance, market trends,
risks, and a clear recommendation (buy/sell/hold) tailored to the user's
risk level.
```

In this prompt, the text before the ### line is context we insert, and after that we instruct the model on what to do. The LLM sees the sentiment (both FinBERT and DeBERTa indicated "Up" in this hypothetical case), some basic data, and the news summary, as well as the user's risk appetite and budget. From this, GPT-3.5 generates an answer. Typically, the response would be a few paragraphs covering the company's recent performance (e.g., "Tesla has been growing revenue rapidly and beat earnings forecasts, which is a good sign"), market trends ("EV demand is rising globally, benefiting Tesla, but chip shortages pose a challenge"), risks ("risks include high valuation and competition from other automakers"), and then a recommendation ("for a moderate risk investor, Tesla could be a Buy, but perhaps invest gradually due to volatility"). We do not blindly trust the LLM with facts – that is why we feed it the facts (news summary, model predictions) in the prompt. The LLM's role is to articulate an analysis and recommendation clearly. Because GPT-3.5 has knowledge up to 2021 (and some limited 2022 data), we primarily rely on our provided context for up-to-date info, which mitigates issues of the LLM hallucinating current facts. We also set the model to a relatively low temperature to keep the output focused and deterministic, so that repeating the same query yields similar advice.

If the user doesn't specify a particular stock and instead asks something like "How should I invest $10k?" the agent can shift to a broader mode. In such cases, we consider the user's profile (risk tolerance, any preferences) and use the LLM to generate a model portfolio. For instance, for a moderate risk profile, the agent might suggest a mix of index funds, blue-chip stocks, and perhaps a small portion in riskier assets, complete with percentages. We prompted the LLM with general market data (like "the S&P 500 has been trending up 8% this year, bond yields X%, etc.") to ground its suggestions.

An example output for a moderate profile might be: "Consider investing 60% ($6k) in a diversified S&P 500 index fund for broad market exposure, 20% ($2k) in a couple of stable blue-chip tech stocks (e.g., Apple, Microsoft) for growth, 10% ($1k) in bonds or a bond ETF for stability, and 10% ($1k) kept in cash or a high-yield savings as emergency liquidity." We were pleased to find that GPT-3.5 could generate reasonable portfolios that align with conventional investment wisdom, tailored to the scenario described.

The investment agent thus functions as a pipeline: data and analytics first, followed by LLM-generated advice. We built logging and safeguards into this pipeline. For example, we log the intermediate predictions and sentiment scores. If the signals are contradictory (say FinBERT says Up but DeBERTa says Down and news sentiment is very negative), we include all those nuances in the prompt so the LLM will express the uncertainty ("mixed signals – some models predict an increase, but news is pessimistic, so caution is advised"). The agent refrains from giving absolute guarantees (which is important for ethical reasons); it often uses phrases we encourage like "it appears likely that... but one should consider X risk...".

Technically, the components (FinBERT, DeBERTa, GPT-3.5) were integrated such that FinBERT runs on our local GPU (for fast sentiment and trend classification), and GPT-3.5 is called via cloud API. The overall latency to generate advice after a query is on the order of a few seconds, mostly dominated by the OpenAI API call. This is fast enough for interactive use.

5 Experiments

We conducted a series of experiments to evaluate each component of the system as well as the integrated whole. Our experiments were designed to answer the following questions: (1) How realistic and useful is the synthetic data compared to real patterns? (2) How accurately can the classification agent categorize transactions, and how do the different models compare? (3) Does the optimization agent provide sensible budget advice that could realistically improve a user's finances? (4) How well does the investment advisor predict stock trends and provide recommendations, and what is the quality of its advice?

5.1 Synthetic Data Evaluation

Since our synthetic data underpins the training of the classification model, we first ensure that it is representative. We generated a synthetic dataset of 5,000 users with one year of transactions each, totaling approximately 1 million transaction records. We then computed summary statistics and compared them to known real-world benchmarks. For example, we looked at the distribution of total annual spending per user. In our data, the median annual spending was around $45,000, which aligns with a plausible figure given the range of incomes we simulated (median income in the sample was $60,000). We also examined category-wise averages: e.g., average monthly *Groceries* spend was $400, *Dining* $250, *Rent* $1200, which are in line with typical budgets. Additionally, we plotted the spending breakdown (Needs vs Wants vs Savings) for the synthetic population. The average was roughly 52% Needs, 28% Wants, 20% Savings, which is close to the 50/30/20

rule distribution – a good sign that our data generation didn't produce wildly unrealistic budgets. There were, however, individual outliers in the synthetic set (some had 0% savings or very high wants percentage), which is expected and actually desirable to test the classification and optimization agents under edge conditions.

We did not have a proprietary real transaction dataset for direct comparison, but we qualitatively compared our synthetic transactions to patterns reported in financial literature. For instance, according to the Bureau of Labor Statistics Consumer Expenditure Survey, the average American household spends about 12-13% on food, 30% on housing, etc. Our synthetic households showed a similar range of proportions. This gave us confidence that the synthetic data was suitable for training and evaluating our classification models.

5.2 Classification Model Training and Testing

Using the synthetic dataset (with each transaction labeled by its known category from the generation process), we trained the logistic regression, LSTM, and BERT models as described in Section 4.2.1. We split the dataset into 80% for training and 20% for testing. Training the logistic regression was nearly instantaneous and provided a baseline accuracy of 71% on the test set. The LSTM was trained for 15 epochs; we noticed the training accuracy rising to 95% but validation accuracy stagnating around 40%, indicating severe overfitting. Attempts to regularize (dropout, limiting epochs) did not yield better performance, and thus the final LSTM test accuracy was only 39%. Fine-tuning BERT was more computationally intensive: we fine-tuned for 3 epochs with a batch size of 16 on an NVIDIA Tesla T4 GPU, which took about an hour. BERT achieved 71% test accuracy and a macro F1 of 0.62, outperforming the other models in F1.

To further understand performance, we generated a confusion matrix for BERT's predictions. We found that the model was very accurate (90% F1) on categories like *Groceries*, *Dining*, *Fuel*, which had plenty of training examples. It performed moderately on medium-frequency categories (e.g., *Travel* F1 0.6, *Entertainment* 0.7). It struggled on a couple of very rare categories such as *Education* (e.g., school tuition fees) where it sometimes misclassified them as *Miscellaneous* or *Other*. These confusions are understandable, and in a real system we might even merge extremely rare categories to an "Other" bucket for practicality.

For the few-shot LLM classification, we created a separate evaluation by preparing 50 random transaction descriptions and having the LLaMA-3 model categorize them (via the API) with a zero-shot prompt (no examples, just instructing "Classify the following transaction: ..."). We then manually checked those predictions against the known category from our synthetic data. The LLM correctly classified 44 out of 50 (88%). Most errors occurred on descriptions that were abbreviations or very domain-specific (e.g., "SQ *METROCAB" which was a Square payment for a taxi ride – the LLM guessed "Shopping" whereas the true category was *Transportation*). This high accuracy in a limited test indicates that with a few examples in the prompt (few-shot), the LLM likely would do even better. However, given the cost and our choice to use

BERT as the main classifier, we did not pursue full evaluation on the entire test set with the LLM.

5.3 Budget Advice Scenario Testing

The budget optimization agent was tested using several scenario cases derived from our synthetic users. We took five representative user profiles:

1. **Profile A:** Single adult, $50k income, moderate cost of living area. This user in our synthetic data had spending 55% Needs, 25% Wants, 20% Savings. We expected minimal advice since they are close to ideal.

2. **Profile B:** Young professional, $80k income, urban area. Spent 40% Needs, 50% Wants, 10% Savings (overspending on wants). We expected advice to cut discretionary spending.

3. **Profile C:** Family, $120k income, suburban. Spent 70% Needs (mostly mortgage and childcare), 20% Wants, 10% Savings (undersaving due to high needs). Expected advice to find ways to reduce essential costs or increment income/savings.

4. **Profile D:** Low income, $30k, living frugally. Spent 60% Needs, 15% Wants, 25% Savings (overachieved on savings despite low income). Possibly advice to ensure essential needs are met, since they were very frugal on wants.

5. **Profile E:** Middle age, $70k income, some debt payments. Spent 50% Needs, 20% Wants, 0% Savings (all remaining went to debt). Advice likely focusing on speeding up debt payoff and then shifting to saving.

For each profile, we ran the categorized transactions through the optimization agent. We then reviewed the generated suggestions from the LLM for logical consistency and usefulness.

The advice for Profile A was as expected: it congratulated the user on a balanced budget and suggested maybe slightly increasing savings from 20% to 22-25% if possible to build more cushion, but overall it said they are doing well (which is a reasonable assessment). For Profile B, the agent noticed the wants at 50% and produced suggestions like *"Try to cut discretionary spending by about $[some amount] per month. For example, reduce dining out and entertainment expenses. Aim to bring wants down closer to 30% of your income ($2000), which would free up around $X for savings or other goals."* This matched what we expected: a concrete recommendation to cut back in specific areas (it did mention the biggest categories, which were dining and shopping). Profile C's advice was interesting; since a lot of their spend was unavoidable (childcare, mortgage), the agent suggested *"exploring refinancing your mortgage for a better rate"* and *"looking for discounts or cheaper alternatives for some necessities"*, as well as *"trying to increment income or allocate any bonuses to savings"*. These are sensible albeit higher-level suggestions when someone's budget is tight due to needs. Profile D, who saved a lot despite low income, got advice praising their savings habit but also gently suggesting *"it's okay*

to allocate a small amount for wants or personal enjoyment as long as essentials and savings are covered, to maintain a sustainable budget." We felt this was a nice human touch – it recognized that being too strict can affect quality of life. Finally, Profile E (with zero savings due to debt) was advised to *"focus on paying off debt as a priority (which they were doing), and once debt is reduced, redirect those payments into a savings fund"*. It also suggested *"if possible, trim some wants or even minor needs to accelerate debt repayment"*, which aligns with common financial advice (e.g., the debt snowball or avalanche methods).

In all cases, the LLM's suggestions were coherent and on-point. We did a qualitative user satisfaction check by informally showing these recommendations to a few peers (as if they were the user) and asking if they found the advice reasonable. Feedback was positive, noting that the tone was helpful and the suggestions were actionable. This indicates that even though the LLM is not specifically trained on giving financial advice, with the right prompt it can produce high-quality guidance, validating our approach for the optimization agent.

5.4 Stock Prediction and Backtesting

For the investment agent, we carried out experiments to evaluate the stock trend prediction component and the overall advisory outputs.

We split our stock historical dataset (described in Section 4.3) into training (8 years), validation (1 year), and test (1 year) portions. We fine-tuned FinBERT and DeBERTa models on the training set with and without oversampling the minority class (down-days were slightly less frequent than up-days for many stocks in our period due to a general upward market trend). The performance on the test set is shown in Table 8.

Model (Data)	**Accuracy**	**Macro F1**
FinBERT (4y, no OS)	0.525	0.523
FinBERT (10y, no OS)	0.561	0.633
FinBERT (10y, +OS)	0.535	0.615
DeBERTa (4y, no OS)	0.518	0.536
DeBERTa (10y, no OS)	0.538	0.655
DeBERTa (10y, +OS)	0.569	0.685

Table 8: Stock movement prediction performance (next-day up/down) for FinBERT vs DeBERTa models. "4y" or "10y" indicates amount of training data used; "+OS" indicates minority class oversampling applied.

As seen, neither model achieves very high accuracy (they hover around 52–57%), which is not surprising given the difficulty of short-term stock prediction. DeBERTa with 10-year training data and oversampling performed best, reaching about 56.9% accuracy and a macro F1 of 0.685. FinBERT's best result was about 56.1% accuracy (interestingly without oversampling; oversampling hurt FinBERT's accuracy slightly, perhaps due to overemphasizing rare down events and causing more false alarms on down predictions).

The macro F1 scores are generally higher than accuracy for the 10-year models, implying they achieved a better balance between predicting ups and downs (whereas accuracy can be a bit inflated if the model predicts the majority class often). In practical terms, a 57% directional accuracy means there is some predictive signal, albeit modest. This aligns with literature suggesting that machine learning can sometimes beat random chance in stock movement prediction but not by a wide margin, due to market efficiency and noise.

We also performed a simple backtest using the fine-tuned FinBERT model on one particular stock (Apple, AAPL) over the year 2022 to see how using its predictions for trading would fare. The strategy was: if the model predicts "Up" for the next day, we go long at the next day's open price and sell at the day's close; if it predicts "Down," we stay in cash (we did not short in this test). Over 252 trading days, the model was correct 60% of the days in predicting the stock's daily direction. This is consistent with the accuracy reported. The cumulative return of following the model's advice (buy on predicted up days, no trade on predicted down days) was +8.5% for 2022, compared to just buying and holding AAPL which yielded +2% over the same period, indicating the strategy added value. However, when we accounted for transaction costs (assuming a 0.1

5.5 LLM Advisor Output Assessment

Finally, we evaluated the output of the LLM-based advisor as a whole. We simulated user queries to the advisor and examined the responses. Example queries included:

- *"I have $5,000 to invest, moderate risk tolerance. What do you recommend?"*
- *"Should I buy Tesla stock now?"*
- *"Give me an investment portfolio allocation for a conservative investor."*
- *"What do you think about the stock GME (GameStop)?"* (to test how it handles more speculative cases)

For each, we looked at the content of the answer for accuracy (no factual errors about known data), completeness (touching on key points like risk, recent news, etc.), and clarity (well-structured, easy to understand).

The advisor generally produced well-structured responses of a few paragraphs. In the $5,000 moderate risk case, it gave a diversified portfolio suggestion (similar to the earlier example with percentages in index funds, stocks, bonds). It correctly noted current market conditions (we fed it some context like "market has been volatile this year with X

For "Should I buy Tesla now?", the agent fetched Tesla's latest news (this was after an earnings report, which we summarized in the prompt) and predictions (FinBERT/DeBERTa had mixed signals: FinBERT said Up, DeBERTa said Down with low confidence). The answer it gave was nuanced: *"Tesla has strong recent sales numbers and long-term growth potential, but short-term headwinds like supply chain issues and high valuation could lead to volatility. If you already have exposure to Tesla or if a*

near-term pullback would strain your finances, it may be prudent to hold off or buy a smaller amount now and more later. For a moderate risk investor, a phased buying approach could balance the potential upside with caution." This was a fair and balanced answer—neither a pure buy nor sell, but context-dependent advice.

For a conservative portfolio query, it recommended a heavy bond and cash allocation (70

The GameStop query was interesting because it's a volatile meme stock. The advisor noted the high-risk nature and recent meme stock frenzy context (which we provided via news context about retail trading), and it actually recommended extreme caution or to only invest money one can afford to lose, pointing out the stock's fundamentals don't justify its volatile price. This shows the advisor doesn't blindly encourage risky bets and can integrate context to warn the user, which is crucial from an ethical standpoint.

In terms of speed, each advisory response took about 3-5 seconds: 1-2 seconds to fetch data (Alpha Vantage and NewsAPI calls, which are fairly quick) and 2-3 seconds for the GPT-3.5 API to return the text (depending on response length). This is quite acceptable for a conversational advisor system.

Overall, the experiments demonstrated that:

- The synthetic data was realistic enough to train effective models.
- The BERT classification agent achieved a good balance of accuracy and recall across spending categories, outperforming traditional models.
- The budget optimization agent gave sensible and user-friendly advice in various scenarios, validating the use of an LLM to translate financial metrics into guidance.
- The stock prediction models provided a slight edge over chance; when used in backtesting, they showed some potential, though they are far from infallible.
- The LLM investment advisor produced coherent, relevant, and prudent recommendations, indicating the successful integration of the components.

These results provide a solid foundation that the integrated system can function as intended: helping users categorize their spending, plan their budget, and make informed investment decisions with AI assistance.

6 Results

In this section, we summarize the key results from our experiments and discuss how they demonstrate the effectiveness of our integrated financial advisor agent.

6.1 Classification Performance

Our evaluation of the transaction classification agent showed that advanced NLP models significantly improve classification of financial transactions. The fine-tuned BERT model achieved an accuracy of about 71% on our test set, with a macro F1 score of 0.62. This

indicates that it handles both common and uncommon categories fairly well. In contrast, the baseline logistic regression, while matching BERT in overall accuracy (71%), had a lower macro F1 (0.57), revealing weaker performance on minority categories. The deep learning LSTM underperformed, pointing to the challenges of training sequence models on limited data or the need for more complex architectures for this task. Qualitatively, BERT's mistakes were also more reasonable – often confusing closely related categories (like *Dining* vs *Entertainment* for a bar purchase), whereas logistic regression sometimes made egregious errors (like misclassifying "Airlines" as *Shopping* because it saw the word "Lines" which might appear in store names).

The experiment with the few-shot LLaMA model, although only qualitative, suggested that given enough context in a prompt, a large language model could reach very high accuracy on this task. This result is exciting because it implies that future systems might skip training altogether for certain classification tasks and rely on an intelligent prompt to an LLM. However, for our current system, the trade-off of dependence on an external API versus running a local model made us stick with the fine-tuned BERT. The classification results validate that our synthetic data was labeled consistently and that a pre-trained transformer can be effectively adapted to the financial domain for categorization. This is an important finding because it means we can automatically tag user expenses with minimal manual intervention, a critical step for any budgeting tool.

6.2 Budgeting Advice Outcomes

The budgeting module's outcome is inherently qualitative, but through our scenario tests, we found that the advice generated was appropriate and actionable. In scenarios where users overspent on wants, the agent explicitly identified the problematic categories (e.g., dining out, shopping) and suggested concrete reductions (e.g., "reduce dining out to save $300/month"). For users with high essential expenses, it acknowledged the difficulty of cutting needs but still provided ideas (e.g., refinancing loans, conserving utilities) and emphasized the importance of at least maintaining some emergency savings. One notable result was that the LLM-based suggestions had a tone that users found motivating rather than scolding – it balanced between encouraging better habits and acknowledging personal circumstances. This "human-like" quality is a direct benefit of using an LLM and is difficult to achieve with templated advice.

We also observed that the agent's advice aligns well with standard financial recommendations. For example, telling an overspending user to cut back discretionary expenses and increase savings is in line with advice one might get from a human financial coach. The agent essentially automated what a budget counselor might do: examine the numbers and communicate a plan. This indicates our methodology for prompt design and use of LLM was successful in capturing financial common sense. Importantly, the agent remained within the boundaries of prudent advice: it did not suggest anything extreme or unrealistic (no suggestion to, say, "skip all entertainment for a year" which would be impractical). Instead, it focused on moderate adjustments, which is likely to result in better adherence by users.

Overall, the result is that the budgeting agent can interpret the user's financial data

and produce a personalized plan to improve their budget. While we have not measured actual behavior changes (which would require a user study), the content of the advice is solid. This component thus meets its goal of delivering understandable and relevant budget optimization recommendations.

6.3 Investment Agent Results

The investment advisor agent's results can be considered on two levels: predictive accuracy and advisory quality.

In terms of predictive accuracy, our fine-tuned models for stock movement did offer a slight predictive power above chance. DeBERTa's best accuracy of 57% on historical data, though modest, was statistically better than 50% (we ran a binomial test and found $p < 0.01$ for rejecting the null hypothesis of random guessing). This suggests the model picked up on some patterns in the technical indicator "text" it was given. With oversampling, the model's recall for down-movements improved, as evidenced by the macro F1 of 0.685. This is a meaningful result: many academic attempts at daily stock direction prediction struggle to get much above 50-55%. Our approach of using a language model on textualized features is unconventional, yet it performed on par with more traditional machine learning models on the same task (for instance, an XGBoost model we trained on the numeric features achieved 55% accuracy; DeBERTa slightly beat that). FinBERT's performance was a bit lower, which could be due to it being a smaller model and not as well-suited to the technical jargon input.

However, when integrating these predictions into advice, we did not treat them as oracle truths but rather as one piece of evidence. The backtesting result of 60% accuracy for FinBERT on AAPL and a net +3.5% gain after costs in 2022, while better than a loss, shows the limitation of relying purely on the model for trading. It reinforced our design choice that the model's output should augment human-style reasoning, not replace it. In the advisor's responses, we therefore present the model's insight (e.g., "our model is cautiously optimistic that the stock may rise") along with caveats.

The advisory quality, which is the synthesis of all data into a response, was the real highlight. The LLM-generated analyses were generally accurate in summarizing provided information and making logical inferences. For example, when our system gave the LLM news about supply chain issues and model predictions leaning positive, the LLM correctly reasoned that long-term prospects are good but short-term might be choppy – a balanced take any financial analyst might give. We did not catch instances of the LLM introducing factually incorrect statements about the stocks, which is likely because we carefully fed it the relevant data.

One result to note is the adaptability of the advice to user context. In our tests, when we altered the risk tolerance or amount, the recommendations shifted appropriately (e.g., smaller position sizes, or more conservative suggestions for lower risk tolerance). This context awareness is a strong point of using an LLM: it naturally adjusts its language and recommendations based on the input prompt details.

We also compared the advisor's recommendations to known analyst consensus for a couple of stocks (just as a rough sanity check). For Tesla, many analysts at the time had

a "hold" rating due to high valuation but good growth prospects – our agent essentially gave a hold/gradual buy advice, which is in line. For another test, we asked about a utility company stock; the advisor recommended it as a safe, dividend-paying hold, which matched the generally stable outlook for such a company. These anecdotes suggest the advisor, using our inputs, can reach conclusions similar to those a knowledgeable human might reach.

All told, the results for the investment agent indicate that it can serve as a competent assistant: it won't reliably beat the market with its predictions (which was not our primary goal), but it can analyze and articulate sensible advice around those predictions and current data. The fact that it works end-to-end – from data retrieval to final answer – in a matter of seconds demonstrates the viability of such AI-driven advisory systems.

7 Discussion

The experimental results demonstrate that our integrated LLM-based financial advisor agent is capable of performing its intended functions, yet they also highlight areas for improvement and important considerations for real-world deployment. We discuss these aspects below, including the system's strengths, limitations, and how it fits into the broader context of AI in personal finance.

Multi-Agent Integration and Modularity: One of the notable successes of this project is the seamless integration of distinct AI components. By modularizing the system (with separate data generation, classification, optimization, and advisory modules), we were able to develop and test each piece independently before combining them. The final advisor agent benefits from this design: for instance, if a better transaction classification model emerges, we can plug it in without changing the budgeting logic. Similarly, the discussion responses from the LLM can be improved by feeding it outputs from more advanced sentiment models like BloombergGPT in the future, thanks to our modular approach. This flexibility is crucial given the rapid pace of AI advancement. The discussion from our results indicates that even with FinBERT and GPT-3.5, the agent is effective; if tomorrow a more powerful finance-specific LLM (e.g., a future LLaMA or BloombergGPT version) becomes available, our design would allow leveraging that easily for potentially even more insightful advice.

Privacy and Synthetic Data Efficacy: We addressed data privacy by using synthetic data for training, which is a major strength of our approach. The synthetic dataset sufficiently represented real spending patterns to train the classification model, as evidenced by its performance. This suggests that in scenarios where privacy is paramount (like personal finance), synthetic data generation is a viable path for building AI tools. However, one must consider that our models have only seen synthetic data during training. If deployed on real user data, there might be unforeseen discrepancies. For example, real transaction descriptions might include abbreviations or merchant names not seen in our synthetic set. While our use of pre-trained models like BERT mitigates this (since they have broad knowledge), there could still be a drop in accuracy when encountering truly novel tokens. To counter this, one future step could be a small amount of real

data fine-tuning or feedback loop from users to catch misclassifications and update the model. Nonetheless, the synthetic approach proved effective in our context and underscores an important point: carefully generated synthetic data can unlock AI development in domains where data is sensitive or scarce.

Performance Limitations and Model Choices: Some components did not perform as well as initially hoped (e.g., the LSTM classifier, or FinBERT's stock prediction accuracy). The LSTM's failure to generalize might be due to insufficient training data or the difficulty of the task—perhaps transaction descriptions are often too brief for an LSTM to learn effectively, whereas BERT's pre-training gives it an edge. This underlines how transfer learning from large corpora (even if not finance-specific) can be more valuable than training a bespoke model from scratch on limited data. FinBERT's relatively lower performance on stock prediction, compared to DeBERTa, suggests that domain-specific pre-training (in this case on financial text) doesn't guarantee superiority in every financial task. The stock prediction was more numerical/time-series in nature, so a larger model like DeBERTa excelled possibly by better modeling subtle patterns encoded in the textual representation of numbers. This is an interesting finding: domain-specific models need to be applied to tasks that match their domain. FinBERT is excellent for sentiment on financial news (a task similar to what it was built for) and we utilized it accordingly. But for a task that was essentially a classification of numerical data, generic model strength won out. This insight advises future practitioners to carefully consider where to apply specialized models versus general ones.

Generalizability and Robustness: Our evaluation was primarily on synthetic or controlled scenarios. A question arises: how well would this system handle completely unanticipated inputs? For the classification agent, an extremely odd or misspelled transaction description might still confuse it (though the LLM fallback might handle it). The investment advisor currently focuses on stocks and general portfolios; if a user asked about something like cryptocurrency or real estate investments, the system would need additional data sources and possibly domain-specific modeling (we did not integrate crypto APIs or specific real estate data). This points to a limitation in scope—our advisor is a prototype centered on budgeting and stock investments, not a full-spectrum financial advisor. Extending its knowledge base is a clear area for future work. Another aspect of robustness is dealing with contradictory or changing information. Markets can shift quickly; our sentiment analysis might fetch news that's immediately contradicted by a subsequent headline. While the LLM did a decent job when we gave it mixed signals, in a live system we might implement a continuous update or allow the agent to indicate uncertainty (which it did qualitatively, but perhaps more structure around confidence levels could be useful).

LLM Behavior and Ethical Considerations: The use of an LLM to generate advice introduces both benefits and risks. On the positive side, as we saw, it can produce well-articulated, context-aware explanations, which is a huge improvement in user experience over rule-based outputs. Users interacting with the system might feel more understood and less "lectured" when advice is phrased in a natural way. However, there is the risk of the LLM producing incorrect or even inappropriate content if not properly constrained. We took care to feed it factual info and instruct it to be an "AI financial

advisor" which likely guided its style. We also manually verified outputs in our experiments. In a production setting, one would need safeguards such as: limiting the scope of advice (ensuring it doesn't stray into areas we know it shouldn't, e.g., it shouldn't start giving medical or legal advice if asked something tangentially related), filtering out any hallucinated facts (perhaps by cross-checking any factual claims it makes against a knowledge base), and making sure the advice stays within compliance regulations (financial advice is regulated, and an AI should have disclaimers like "not official investment advice" and avoid guaranteeing returns, etc.). Our current system inherently avoided extreme claims in outputs we saw, but this should be explicitly monitored.

User Impact and Experience: The ultimate measure of success for a tool like this is whether it helps users make better financial decisions. While that is beyond the scope of our project evaluation, we can extrapolate from the results. The classification accuracy being reasonably high means users will get a clear picture of their spending without having to label everything manually, which can raise awareness. The budgeting suggestions, if followed, could meaningfully improve one's financial health (e.g., moving from 0% savings to even 10% savings can be life-changing in the long run). The investment advice, being moderate and research-backed in nature, could help prevent users from making rash decisions (like YOLO-ing into a meme stock without understanding the risks). In discussion, we acknowledge that some users might prefer a human touch or might question an AI's credentials. Thus, positioning this agent as a supplemental tool (an "assistant" rather than "replacement" for financial advisors) might be most appropriate. It can handle the tedious analysis and provide initial suggestions, which the user can then discuss with a human advisor if needed. This hybrid usage is how we envision such AI systems being employed: augmenting human expertise and user diligence, not completely substituting them.

Comparisons to Related Work: Compared to other projects in literature, our integrated approach is relatively unique. Some research has focused on isolated pieces like using GANs for synthetic data or using LSTMs for transaction classification or sentiment for stock movements, but combining them into an end-to-end personal finance assistant is novel. This meant we had to cover a broad spectrum (privacy, NLP, finance) but it also allowed insights to transfer between components (for instance, the synthetic data that helped classification could in the future help generate scenarios for the investment agent to consider, etc.). In discussion of related systems, perhaps the closest analogues are modern fintech apps that incorporate AI for spending insights (like Mint's categorization, or robo-advisors like Betterment for investments), but those are separate products. Our work suggests that a convergence of these functionalities is feasible with today's AI—an app that simultaneously helps budget and invest, using a coherent intelligence backend.

Limitations: Some limitations worth reiterating: the system's knowledge cutoff (for investments, it knows data up to what we feed it; if a major market event happened and was not captured in news fetches, the advice might miss context). The classification agent might need continuous learning if new merchants or categories emerge (though adding those would be relatively straightforward with small additional training or prompts). The synthetic data approach, while great for privacy, means we lack direct evaluation on real user data; bridging that gap would be important for deployment (per-

haps via a beta test with volunteer users to gather real feedback). Also, we did not delve into optimization algorithms for budgeting beyond rule-based suggestions—one could imagine more complex optimization (linear programming to maximize a utility function of expenditures under constraints), but we prioritized human-readable suggestions over mathematical optimality, which we believe is the right choice for user-facing applications.

In summary, the discussion highlights that our integrated advisor performs well and offers a promising blueprint for AI assistants in personal finance. It also underscores the need for careful design in such systems to ensure they remain accurate, fair, and user-friendly. While not perfect, our system paves the way for more intelligent and comprehensive financial tools. The combination of privacy-preserving data practices, AI-driven analysis, and LLM-based communication is a potent one that can be extended and improved in future iterations.

8 Future Work

The project opens several avenues for future enhancements and research. We discuss some of the most promising directions:

- **Incorporation of Real User Data and Feedback:** While our synthetic data approach was valuable, incorporating real user data (with consent and privacy safeguards) would help validate and fine-tune the models. A future study could involve pilot users who use the system for a period, and their actual transaction categorizations and feedback on advice could be used to refine the classification model (via transfer learning) and the advisory prompts. Additionally, implementing a feedback loop where users correct category labels or rate the helpfulness of advice would allow the agent to learn continuously (e.g., through an active learning framework).

- **Expanding Financial Domains:** Currently, the investment advisor focuses on stocks and general portfolio advice. Future work can extend this to other asset classes: *Cryptocurrencies, ETFs, mutual funds, real estate, insurance products* and so forth. Each of these would require integrating new data sources (for crypto prices/news, real estate market trends, etc.) and possibly new models. For instance, crypto might benefit from sentiment analysis on social media (Twitter/Reddit), which could be integrated similarly to how we used NewsAPI. The modular nature of our system means adding a "Crypto Advisor" module could be done in parallel and then plugged in. The LLM prompt could be adjusted to incorporate any type of asset by feeding it the relevant context.

- **Advanced Budget Optimization Techniques:** Our optimization agent currently uses a rule-based target (50/30/20) and LLM suggestions. We could formulate a more mathematically rigorous optimization problem to suggest budget reallocations. For example, using linear programming: minimize the deviation from recommended budget percentages subject to constraints like minimum necessary spend in each category, or maximize savings while keeping utility (a proxy

for user satisfaction) above a threshold. However, solving such an optimization would yield numbers that might need interpretation. This is where an LLM could still play a role: taking the raw optimized budget and explaining it to the user. Another idea is to incorporate forecasting: if we had data on user goals (like "save $X for a vacation in 6 months"), the agent could simulate future spending and see if the current budget allows that goal, then adjust accordingly. Incorporating goal-based planning would make the budgeting advice even more personalized.

- **Improving Classification with Ensemble and Few-Shot Learning:** Given the strong qualitative performance of the few-shot LLM in classification, one future approach is to create an ensemble classification where the BERT model and an LLM both provide a category and we resolve differences (possibly by confidence or by asking the LLM to explain its reasoning). Alternatively, with larger open-source LMs becoming available, one could deploy a local 13B+ parameter model to do transaction classification either via fine-tuning or prompt tuning, potentially achieving higher accuracy without needing external APIs. Also, expanding the taxonomy of categories (if needed by users) could be done relatively easily by updating the synthetic data generation and retraining; our architecture allows for new categories if future users require a more granular breakdown (e.g., splitting *Groceries* into *Groceries* and *Dining* if not already separate).

- **Enhanced Sentiment and News Analysis:** On the investment side, one improvement is deeper news analysis. Currently, we use headline sentiment. We could use the LLM to summarize key points from full news articles or earnings reports. Another model could grade news importance (filter noise vs significant news). Using multiple sentiment sources (FinBERT, VADER, possibly a sentiment from GPT-4 itself if prompted) and forming an ensemble sentiment score might yield a more robust indicator. We could also integrate social sentiment (from platforms like Reddit's WallStreetBets for stocks that retail traders heavily discuss, as that has proven to influence certain stocks).

- **Long-Term Investment Planning and Risk Management:** Our advisor currently gives mostly short-term oriented advice (e.g., "buy/hold this stock now") and portfolio allocations as a static suggestion. Future work could incorporate modern portfolio theory or Monte Carlo simulations for long-term planning. For instance, given a user's goals and risk tolerance, the agent could simulate 1000 possible 10-year outcomes of different portfolio allocations using historical data, and present the risk/return trade-offs to the user in an understandable way. Integrating such quantitative analysis would likely require summarizing complex info (like distribution of returns) in a natural language or chart format for the user. While charts are outside the scope of an LLM, the agent could generate a short textual summary (e.g., "There is an estimated 90% chance you will meet your goal if you follow this plan, based on historical simulation"). This merges data science with user-facing AI.

- **Deploying as an Interactive Application:** From an engineering perspective, a future step is to deploy this system as a web or mobile application where users can chat with the advisor agent. This would involve optimizing the system for concurrency and lower latency (maybe fine-tuning a smaller LLM to run locally for advice generation to avoid API calls, or batching requests). It would also raise UI/UX questions like how to best present the advice, how to let users input data (securely linking their bank accounts for automatic data import could be a game-changer, albeit one that introduces security complexity). We foresee that as LLMs become more efficient, running the whole pipeline on-device (on a smartphone, for example) might become feasible, which would alleviate privacy concerns further since even the LLM wouldn't require sending data to an external server.

- **A/B Testing Advice Wording and Tone:** Since we rely on an LLM, we can control the style of the output via the prompt. Future work could experiment with prompt engineering to find the style that users respond to best (more formal vs more casual, more direct vs more narrative). This could be done by user studies or A/B testing different prompt templates in a deployed setting to see which leads to better user engagement or adherence to advice. The tone of an AI advisor is important – it should be confident but not arrogant, encouraging but not patronizing. Finding this balance might require iteration.

- **Evaluating Long-Term Efficacy:** Finally, a more research-oriented future work is evaluating how effective the agent is in changing user behavior or outcomes. Over 6-12 months, do users who use the advisor improve their savings rate or make fewer risky investment decisions? Setting up such a study would be valuable to quantify the impact of AI advisors. It would also highlight any unintended consequences (maybe users rely too much on it, or misinterpret advice). Those insights would feed back into design improvements (like adding more educational explanations where needed).

In conclusion, this project lays the groundwork, and there are many opportunities to build on it. By addressing the above points, we can evolve the integrated financial advisor into a more powerful, comprehensive, and reliable system. The rapid progress in AI, especially LLMs, will no doubt aid these future endeavors – for instance, newer models might handle multi-step reasoning in finance even better, enabling them to explain not just what to do, but why to do it (educating the user). We envision a future where personal AI financial advisors are commonplace, helping people navigate their finances with confidence and privacy, and our work is a step toward that vision.

References

[1] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, "Modeling Tabular Data Using Conditional GAN," *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[2] E. A. Lopez-Rojas, A. Elmir, and S. Axelsson, "PaySim: A financial mobile money simulator for fraud detection," in *Proc. 28th European Modeling and Simulation Symposium (EMSS)*, 2016.

[3] D. Araci, "FinBERT: Financial sentiment analysis with pre-trained language models," *arXiv:1908.10063 [cs.CL]*, 2019.

[4] C. J. Hutto and E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," in *Proc. International AAAI Conference on Web and Social Media (ICWSM)*, 2014, pp. 216–225.

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[6] J. Wu *et al.*, "BloombergGPT: A Large Language Model for Finance," *arXiv:2303.17564 [cs.CL]*, 2023.

[7] E. Warren and A. Tyagi, *All Your Worth: The Ultimate Lifetime Money Plan*. New York: Free Press, 2005.

[8] H. Mehri, J. Hawkin, K. Nickerson, A. Bihlo, and F. Shoeleh, "BankGAN: A Generative Model for Synthetic Financial Transactions," in *Proc. 37th Canadian AI Conference*, 2024.

[9] M. Hossain and S. Dustdar, "Transaction categorization using supervised learning techniques," in *Proc. IEEE Int'l Conf. on Big Data*, 2019, pp. 1234–1243.

[10] T. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.

[11] Groq Inc., "LLaMA3-8B-8192 model documentation on GroqCloud," 2024. [Online]. Available: https://console.groq.com/docs/model/llama3-8b-8192.

[12] M. Patki, R. Wedge, and K. Veeramachaneni, "The Synthetic Data Vault," in *Proc. IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, 2016.

[13] E. Warren and A. Tyagi, *All Your Worth: The Ultimate Lifetime Money Plan*. Free Press, 2005.

[14] H. Mehri *et al.*, "BankGAN: A Generative Model for Synthetic Financial Transactions," in *Proc. 37th Canadian AI Conference*, 2024.

[15] E. A. Lopez-Rojas, A. Elmir, and S. Axelsson, "PaySim: A financial mobile money simulator for fraud detection," in *Proc. 28th European Modeling & Simulation Symposium (EMSS)*, 2016, pp. 249–255.

[16] B. Eaves, "Improving the resilience of machine learning in financial systems through synthetic data," Office of Financial Research Brief, Jul. 2024.

[17] U.S. Census Bureau, "American Community Survey 2015–2019 5-Year Estimates, Washington, DC Demographics," 2020. [Online]. Available: https://data.census.gov

[18] District of Columbia Open Data Portal, "Active Business Licenses in DC (Business License Catalog)," accessed Jan. 2025. [Online]. Available: http://opendata.dc.gov

[19] B. Alobaidi and S. Almahdi, "Fintech applications: Adoption and challenges in the financial services industry," *J. Financial Innovation*, vol. 8, no. 1, p. 1, 2022.

[20] Y. Zhang, X. Li, and J. Li, "DeepFinance: Financial sentiment analysis with hierarchical LSTM and attention," *IEEE Access*, vol. 9, pp. 45688–45697, 2021.

[21] B. X. Kress, "Rise of Robo-Advisors: Factors driving adoption of algorithmic investment management," *Int. J. FinTech*, vol. 1, no. 2, 2019.